

Complexity Analysis of a SymPy-Based Fitness Function for Metaheuristic Benchmarking

Computational Physics & Optimization Group

June 4, 2026

Abstract

We analyze the computational complexity of a fitness function derived from a symbolic potential model (implemented in SymPy) that is used within metaheuristic (MH) algorithms. The fitness evaluation consists of a fixed number of arithmetic operations and is therefore $O(1)$ per call. For a family of metaheuristics under a fixed budget of evaluations, the total asymptotic complexity is linear in the number of evaluations, with constant factors that depend on the MH's internal overhead. We provide a systematic measurement methodology and derive a general Big O expression that can be applied across different optimizers. This work is part of a No Free Lunch (NFL) style experimental study.

1 Introduction

In metaheuristic optimization, the cost of evaluating the fitness function often dominates total runtime. However, for symbolic models with a fixed number of variables, each evaluation takes constant time. We examine a concrete implementation: a fitness function that sums four components (potential, Hessian trace, gradient modulus, and tachyon level), each derived from a two-field scalar potential $V_{\text{lift}}(s, \tau; \vec{\alpha})$ where $\vec{\alpha} = (AH3, AF3, AF5, A3N3, AD5)$ are real parameters. The function is written in SymPy and then lambdified to NumPy/CuPy for numerical evaluation.

2 Complexity of the Fitness Function

Let $f(\vec{\alpha})$ be the fitness function:

$$f(\vec{\alpha}) = V_{\text{lift}}(\vec{\alpha}) + \text{tr}(\mathbf{H}) + \|\nabla V_{\text{lift}}\| + \lambda_{\min}(\mathbf{H})$$

with $\mathbf{H}$ the 2×2 Hessian matrix. All terms are closed-form algebraic expressions obtained by symbolic differentiation. The lambdified code performs a fixed number of floating-point operations (additions, multiplications, divisions, powers, square roots) that does not depend on any input size. Therefore:

$$T_{\text{fitness}}(1) = O(1) \quad (\text{constant time}).$$

Let C_f be the measured average time per evaluation (a constant). For E evaluations, total fitness cost is $C_f \cdot E = O(E)$.

3 Metaheuristic Complexity: General Form

Let a metaheuristic $\mathcal{M}$ be characterized by:

- Population size (or number of agents) P ,
- Number of generations (iterations) G ,
- Total number of fitness evaluations $E = P \times G$ (for generational algorithms; some MHs may use different relations).

The total runtime of $\mathcal{M}$ is:

$$T_{\mathcal{M}} = \underbrace{C_f \cdot E}_{\text{fitness}} + T_{\text{int}}(P, G)$$

where $T_{\text{int}}(P, G)$ is the internal overhead (selection, crossover, mutation, velocity updates, etc.). For typical MHs, $T_{\text{int}} = O(E \cdot h(P))$ with $h(P)$ usually $O(1)$ or $O(\log P)$. Hence asymptotically:

$$T_{\mathcal{M}} = O(E) \quad \text{with respect to problem size (fixed).}$$

3.1 Dependence on the Metaheuristic

For a fixed evaluation budget E_{max} (e.g., 10^5), all compared MHs perform exactly the same number of fitness calls. Thus the asymptotic complexity is identical for all; only the constant factors differ:

$$T_{\mathcal{M}} = E_{\text{max}} \cdot C_f + \text{constant}_{\mathcal{M}} \cdot E_{\text{max}}$$

where $\text{constant}_{\mathcal{M}}$ captures the per-evaluation overhead of the MH's internal operations. Therefore in an NFL-style experiment, the measured performance differences reflect the algorithms' efficiency (including constant factors), not asymptotic growth differences.

4 Systematic Measurement Methodology

To empirically determine the complexity parameters for a specific MH:

1. Measure C_f :

```
import time
N = 10000
t0 = time.perf_counter()
for _ in range(N):
    fitness_function_fixedmoduli_lift(AH3, AF3, AF5, A3N3, AD5)
t1 = time.perf_counter()
C_f = (t1 - t0) / N
```

2. Run the MH with varying E : Keep P fixed and increase G (or vice versa) so that $E = P \cdot G$ varies linearly.

3. Measure total wall time T_{total} .

4. Subtract the fitness cost:

$$T_{\text{int}} = T_{\text{total}} - C_f \cdot E$$

5. Perform regression: Plot T_{total} vs E ; the slope is $C_f + \text{const}_{\mathcal{M}}$ and the intercept captures constant initialization overhead.

4.1 Example: Genetic Algorithm (GA)

For a GA with population P , generations G , tournament selection of size k , one-point crossover, and bit-flip mutation:

- Fitness evaluations: $E = P \cdot G$
- Sorting population (for elitism): $O(P \log P)$ per generation
- Selection: $O(P \cdot k)$ per generation
- Crossover/Mutation: $O(P)$ per generation

Hence $T_{\text{int}} = G \cdot O(P \log P + Pk) = O(E \log P)$. For fixed P , this is $O(E)$; for variable P , the $\log P$ factor may appear.

5 Conclusion

The fitness function derived from the two-field scalar potential is $O(1)$ per evaluation. Consequently, for any metaheuristic operating under a fixed evaluation budget, the total complexity is linear in that budget. The constant factor is the sum of the per-evaluation fitness cost C_f and the per-evaluation internal overhead of the algorithm. This linearity enables fair comparisons in NFL-style benchmarks: differences in wall-clock time directly reflect algorithmic efficiency modulo the constant C_f , which is identical across all methods.

References

- [1] Cormen, T. H. et al. *Introduction to Algorithms*. MIT Press, 2009.
- [2] Wolpert, D. H., Macready, W. G. “No Free Lunch Theorems for Optimization”. *IEEE Trans. Evol. Comput.* 1(1), 1997.
- [3] Meurer, A. et al. “SymPy: symbolic computing in Python”. *PeerJ Computer Science* 3, 2017.